参考:

1. 深入浅出pytorch_哔哩哔哩_bilibili

- 一.Tensor
  - 1.张量属性
  - 2.张量创建
    - 2.1.直接创建Tensor
    - 2.2. 依据数值创建Tensor
    - 2.3.依据分布创建 Tensor
  - 3.张量操作
    - 3.1张量拼接
    - 3.2张量切分
    - 3.3张量索引
    - 3.4张量变换
  - 7.张量运算
    - 7.1四则运算
    - 7.2特殊四则运算
    - 7.3指对幂运算
    - 7.4三角运算
    - 7.5其他运算
- 二.Computational Graph
  - 1.Computational Graph
  - 2.Grad Chain
  - 3.Dynamic Graph
  - 4.AutoGrad
- 三.torchvision
  - 1.datasets
  - 2.transforms
  - 3.model
- 四.Data
  - 1.Dataset
  - 2.DataLoader
  - 3.Transforms
- 五.Module
  - 1.Layer
    - 1.1卷积层
    - 1.2池化层
    - 1.3全连接层
    - 1.4激活函数层
    - 1.5初始化参数
  - 2.Container
    - 2.1.torch.nn.Sequetial
    - 2.2.torch.nn.ModuleList
    - 2.3.torch.nn.ModuleDict
  - 3.Block

    - 4.torch.nn.Module类
- 六.Loss Function & Optimizer & LRScheduler
    - 1.Loss Function
    - 2.Optimizer
    - 3.LRScheduler
- 七.TensorBoard
- 八.Hook & CAM
    - 1.Hook
    - 2.CAM
- 十.Regularization
    - 1.weight decay(权值衰减)
    - 2.Dropout(随机失活)
    - 3.Normalization(标准化)
- 十一.Transfer Learning
    - 1.保存与加载
    - 2.断点续训练
    - 3.模型微调(Model Finetune)
- 十二.GPU
    - 1.CPU & GPU
    - 2.转移设备
    - 3.多GPU并行运算
- 十三.Apply
    - 1.图像分类
    - 2.图像分割
    - 3.目标检测
    - 4.生成对抗网络
    - 5.循环神经网络

一.Tensor

1.张量属性

- data: 被包装的数据
- dtype: 张量的数据类型
- shape: 张量的形状
- device: 张量所在的设备,是加速的关键
- grad: data的梯度
- grad_fn: 创建Tensor的Function,是自动求导的关键
- requires_grad: 指示是否需要梯度
- is_leaf: 指示是否是叶子结点(张量)

2.张量创建

2.1.直接创建Tensor

1. 从data创建: torch.tensor(data)
2. 从numpy.ndarray创建: torch.from_numpy(ndarray)

   注意: 此时tensor将与ndarray共享内存

2.2. 依据数值创建Tensor

1. 全0张量: torch.zeros(size)
2. 全1张量: torch.ones(size)
3. 全n张量: torch.full(size,fill_value)

2.3.依据分布创建 Tensor

1. 一维等差张量: torch.arange(start=0,end,step=1)
2. 一维均分张量: torch.linspace(start,end,steps=100)
3. 一维对数均分张量: torch.logspace(start,end,steps=100,base=10.0)
4. 二维单位对角张量: torch.eye(n,m=None)
5. 正态分布: torch.normal(mean,std,size=None)

   依据mean和std类型(标量,张量)不同有四种不同形式
6. 标准正态分布: torch.randn(size)

   mean=0,std=1的正态分布称为标准正态分布
7. 区间[0,1)浮点数均匀分布: torch.rand(size)
8. 区间[low,high)整数均匀分布: torch.randint(low=0,high,size)
9. 随机排列: torch.randperm(n)

   生成从0到n-1的随机排列
10. 0-1分布: torch.bernoulli(imput)

    以input为概率生成0-1分布

3.张量操作

3.1张量拼接

1. 按dim拼接: torch.cat(tensors,dim=0)
2. 在新创键的dim拼接: torch.stack(tensors,dim=0)

3.2张量切分

1. 按dim平均切分为chunks份: torch.chunk(input,chunks,dim=0)
2. 按dim指定split_size_or_sections切分: torch.split(tensor,split_size_or_sections,dim=0)

   split_size_or_sections为int时表示每份长度;为list时按list切分

3.3张量索引

1. 提取dim的index进行拼接: torch.index_select(input,dim,index)
2. 按mask中的True返回一维张量: torch.mark_select(input,mask)

3.4张量变换

1. 变形: torch.reshape(input,shape)

   新形状的张量与原张量共享内存

2. 交换维度: torch.transpose(input,dim0,dim1)
3. 二维转置: torch.t(input)

   等价于torch.transpose(input,0,1,)

4. 压缩长度为1的dim: torch.squeeze(input,dim=None)

   dim为None时移除所有长度为1的维度;否则移除指定维度

5. 扩展维度: torch.unqueeze(input,dim)

7.张量运算

7.1四则运算

1. torch.add()
2. torch.sub()
3. torch.mul()
4. torch.div()

7.2特殊四则运算

1. torch.addcmul()
2. torch.addcmul()

7.3指对幂运算

1. torch.exp()

2. torch.pow()
3. torch.log()
4. torch.log2()
5. torch.log10()

7.4三角运算

1. torch.cos()
2. torch.acos()
3. torch.cosh()
4. torch.cos2()

7.5其他运算

1. torch.abs()

二.Computational Graph

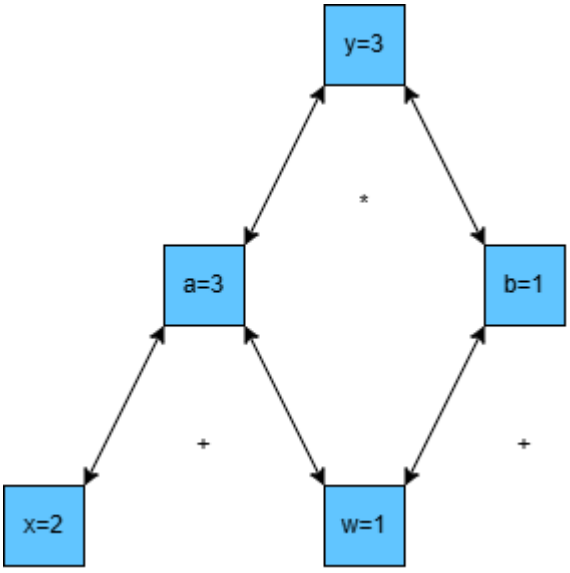
1.Computational Graph

例如$y=(x+w)*(w+1)$

可以拆分为$a=x+w$, $b=w+1$, $y=a*b$

此时Tensor为节点,Tensor.is_leaf表示是否为叶子节点(自变量,例如x和w),

Tensor.grad_fn:为创建Tensor的Function(例如图中的+和*)



2.Grad Chain

$dy/dx=dy/da*da/dx$

$dy/dw=dy/da*da/dw+dy/db*db/dw$

借助计算图能快速有效地进行链式求导

3.Dynamic Graph

Pytorch动态图:计算与搭建**同时**进行,灵活易调节

TensorFlow静态图:**先**搭建图**后**计算,高效不灵活

4.AutoGrad

1. 自动求取梯度: backward()

```
torch.autograd.backward(
    tensors,  # 用于求导的张量,如loss
    grad_tencors=None,  # 多梯度权重
    retain_graph=None,  # 创建导数计算图,用于高阶求导
    create_graph=None,  # 保存计算图
)
```

2. 求取梯度: grad()

```
torch.autograd.backward(
    outputs,  # 用于求导的张量,如loss
    inputs,  # 需要梯度的张量
    grad_tencors=None,  # 多梯度权重
    retain_graph=None,  # 创建导数计算图,用于高阶求导
    create_graph=None,  # 保存计算图
)
```

梯度不自动清零,请调用zero()

依赖叶子节点的节点,requires_grad默认为True

叶子节点不可执行in_place,求梯度时会用到原值

三.torchvision

计算机视觉工具包

1.datasets

常用的数据集的dataset实现,MNIST,CIFAR-10,ImageNet等

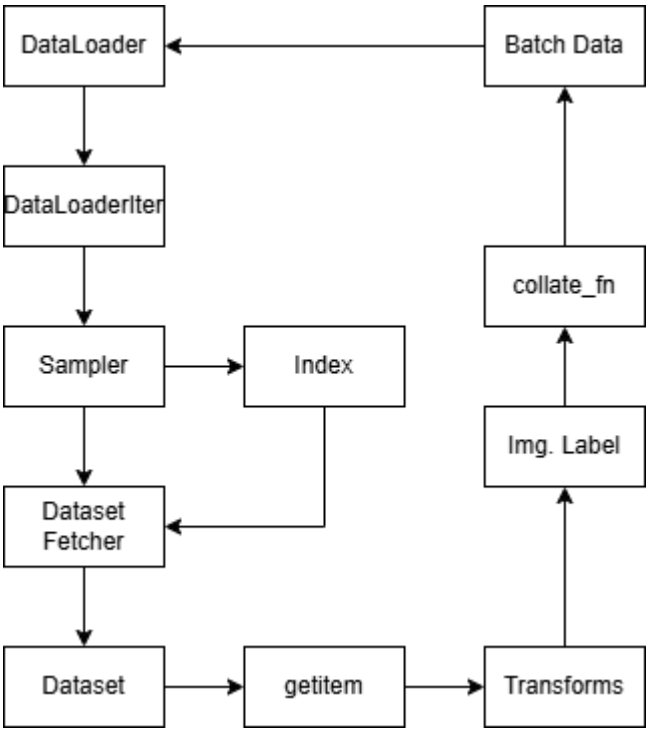
2.transforms

常用的图像预处理方法

3.model

常用的模型预训练,AlexNet,VGGNet,GoogLeNet,ResNet

四.Data



1.Dataset

Dataset抽象类,所有自定义的Dataset需要继承它并复写`__getitem__()`

`__getitem__()`接受一个索引,返回一个样本

2.DataLoader

```
DataLoader(
    dataset,  # 数据
    batch_size,  # 批大小
    shuffle=False,   # 打乱每个epoch
    sampler=None,  # 抽取策略
    batch_sampler=None,  # 直接抽取批
    num_workers=0,  # 多进程
    collate_fn=None,  # 合并策略
    pin_memory=False,  # cuda固定内存
    drop_last=False,  # 丢弃最后一批不完整的数据
    timeout=0,  # 加载数据的超时时间
    worker_init_fn=None,  # 初始化函数
    multiprocessing_context=None  # 多进程上下文
)
```

3.Transforms

标准化与数据增强



1. 数据中心化
2. 数据标准化
3. 裁剪
4. 填充
5. 遮挡
6. 噪声添加
7. 灰度化
8. 线性变换
9. 仿射变换(旋转,平移,错切,缩放,翻转)
10. 亮度,饱和度,对比度,色相
11. 随机操作
12. 自定义Lambda
13. 自定义Transforms

五.Module

1.Layer

1.1卷积层

1. 下采样

```
torch.nn.Conv2d(
    in_channels,  # 输入通道数
    out_channels,  # 输出通道数
    kernel_size,  # 卷积核尺寸
    stride=1,  # 步长
    padding=0,  # 填充
    dilation=1,  # 空洞卷积大小
    groups=1,  # 分组卷积设置
    bias=True,  # 偏置
    padding_mode='zeros'
)
```

输出尺寸:

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

2. 上采样(转置卷积)

```
torch.nn.ConvTranspose2d(
    in_channels,  # 输入通道数
    out_channels,  # 输出通道数
    kernel_size,  # 卷积核尺寸
    stride=1,  # 步长
    padding=0,  # 填充
    dilation=1,  # 空洞卷积大小
    groups=1,  # 分组卷积设置
    bias=True,  # 偏置
    padding_mode='zeros'
)
```

输出尺寸:

$$H_{out} = (H_{in} - 1) \times \text{stride}[0] - 2 \times \text{padding}[0] + \text{dilation}[0] \times (\text{kernel_size}[0] - 1) + \text{output_padding}[0] + 1$$

1.2池化层

对信号进行收集和总结

1. 平均池化

```
torch.nn.AvgPool2d(
    kernel_size,  # 池化核尺寸
    stride=None,  # 步长
    padding=0,  # 填充
```

```
    count_include_pad=True,  # 填充值用于计算
    divisor_override=None,  # 除法因子
    cell_mode=False  # 记录池化像素索引
)
```

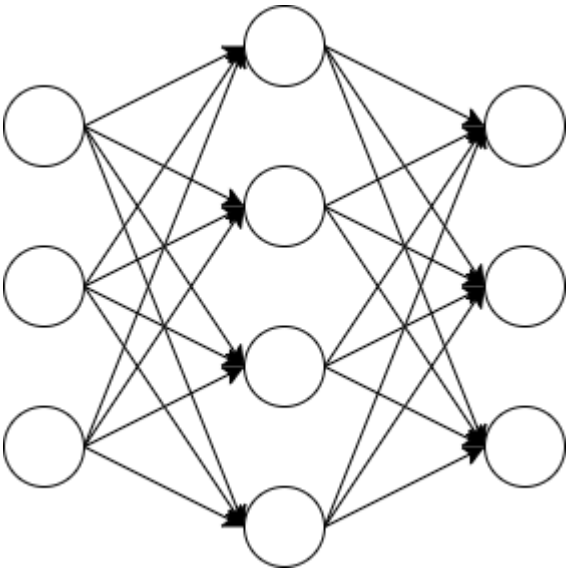
2. 最大池化

```
torch.nn.MaxPool2d(
    kernel_size,  # 池化核尺寸
    stride=None,  # 步长
    padding=0,  # 填充
    dilation=1,  # 间隔
    return_indices=False,  # 尺寸向上取整
    cell_mode=False  # 记录池化像素索引
)
```

3. 上采样池化

```
torch.nn.MaxUnPool2d(  # 以Max为例
    kernel_size,  # 池化核尺寸
    stride=None,  # 步长
    padding=0,  # 填充
)
```

1.3全连接层



1.4激活函数层

后续补充每个激活函数详情:图像,公式,梯度,特点

1. Sigmoid()

2. tanh()
3. Softmax
4. ReLU()
5. LeakyReLU()
6. PReLU
7. RReLU

1.5初始化参数

后续补充每个初始化方法详情:函数,简介

1. Xavier均匀分布
2. Xavier正态分布
3. Kaiming均匀分布
4. Kaiming正态分布
5. 均匀分布
6. 正态分布
7. 常数分布
8. 正交矩阵
9. 单位矩阵
10. 稀疏矩阵

2.Container

2.1.torch.nn.Sequetial

顺序性,各网络层之间严格按顺序执行,常用于block构建

2.2.torch.nn.ModuleList

迭代性,常用于大量重复网构建,通过for循环实现重复构建

append():添加

extend():拼接

insert():插入

2.3.torch.nn.ModuleDict

索引性,常用于可选择的网络层

clear():清空

items():键值对

keys():键

values():值

pop():删除

3.Block

略

4.torch.nn.Module类

```
self._parameters  # 存储管理nn.Parameter类,可学习参数如weight,bias
self._modules  # 存储管理nn.Module类
self._buffers  # 存储管理缓冲属性
# 存储管理钩子函数
self._backward_hooks
self._forward_hooks
self._forward_pre_hooks
self._state_dict_hooks
self._load_state_dict_pre_hooks
```

六.Loss Function & Optimizer & LRScheduler

1.Loss Function

1. **torch.nn.CrossEntropyLoss()**

   torch.nn.LogSoftmax()与torch.nn.NLLLoss()结合,进行交叉熵计算

2. torch.nn.NLLLoss()

   实现负对数似然函数中的负号功能

3. torch.nn.BCELoss()

   二分类交叉熵

4. torch.nn.BCEWithLogLoss()

   结合Sigmoid与二分类交叉熵

5. torch.nn.L1Loss()

   计算inputs与target之差的绝对值

6. **torch.nn.MSELoss()**

   计算inputs与target之差的平方

7. torch.nn.SmoothL1Loss()

   平滑的torch.nn.L1Loss()

8. torch.nn.PoissonNLLLoss

   泊松分布的负对数似然损失函数

9. torch.nn.KLDivLoss()

   计算KL散度divergence相对熵

10. torch.nn.MarginRankingLoss()

    计算两个向量之间的相似度,用于排序任务

11. torch.nn.MultiLabelMarginLoss()

    多标签边界损失函数

12. torch.nn.SoftMarginLoss()

    计算二分类的logistic损失

13. torch.nn.MultiLabelSoftMarginLoss()

    torch.nn.SoftMarginLoss()的多标签版本

14. torch.nn.MultiMarginLoss()

    计算多分类的折页损失

15. torch.nn.TripletMarginLoss()

    计算三元组损失,人脸验证中常用

16. torch.nn.HingeEmbeddigLoss()

    计算两个输入的相似性,常用于非线性embedding和半监督学习

17. torch.nn.CosineEmbeddigLoss()

    采用余弦相似度计算两个输入的相似性

18. torch.nn.CTCLoss()

    计算CTC损失,解决时序类数据的分类

2.Optimizer

1. **torch.optim.SGD**

随机梯度下降法

2. torch.optim.Adagrad

   自适应学习率下降法

3. torch.optim.RMSprop

   torch.optim.Adagrad的改进

4. torch.optim.Adadelta

   torch.optim.Adagrad的改进

5. **torch.optim.Adam**

   torch.optim.RMSprop结合Momentum

6. torch.optim.Adamax

   torch.optim.Adam增加学习率上限

7. torch.optim.SparseAdam

   稀疏版的Adam

8. torch.optim.ASGD

   随机平均梯度下降

9. torch.optim.Rprop

   弹性反向传播

10. torch.optim.LBFGS

    torch.optim.BFGS的改进

3.LRScheduler

```
class _LRScheduler():  # LRScheduler类
    def __init__(
    self,
    optimizer,  # 关联的优化器
    last,  # 记录epoch数
    base_lrs=-1,  # 记录初始学习率
)

    def get_lr()  # 计算下一个epoch学习率

    def step()  # 更新下一个epoch的学习率
```

1. 等间隔调整学习率StepLR()

```
torch.lr_scheduler.StepLR(
    step_size: 调整间隔数
    gamma=0.1: 调整系数
)  # lr=lr*gamma
```

2. 给定间隔调整学习率MultiStepLR()

```
torch.lr_scheduler.MultiStepLR(
    milestones: 设定调整时刻数
    gamma=0.1: 调整系数
)  # lr=lr*gamma
```

3. 指数衰减调整学习率ExponentialLR()

```
torch.lr_scheduler.ExponentialLR(
    step_size: 调整间隔数
    gamma=0.1: 指数的底
)  # lr=lr*gamma**epoch
```

4. 余弦周期调整学习率CosineAnnealingLR()

```
torch.lr_scheduler.CosineAnnealingLR(
    T_max: 下降周期
    eta_min=0: 学习率下限
)  # lr=eta_min+0.5*(base_lrs-eta_min)(1+cos(pai*epoch/T_max)
```

5. 监督指标,当指标不再变化则调整ReduceLRonPlateau()

```
torch.lr_scheduler.ReduceLRonPlateau(
    mode='min',  # min/max两种模式
    factor=0.1,  # 调整系数
    patience=10,  # 接受几次不变化
    cooldown=0,  # 停止监控一段时间
    verbose=False,  # 是否打印日志
    min_lr=0,  # 学习率下限
    eps=1e-08,  # 学习率衰减最小值
)
```

6. 自定义调整Lambda()

七.TensorBoard

1. 安装

   cmd: `pip install tensorboard futrue`

2. 记录

```
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter(
    log_dir=None,  # 输出文件夹
    comment='',  # 不指定log_dir时,文件夹后缀
    filename_suffix,  # 文件名后缀
)

writer.close()
```

3. 数据

   - 标量

```
for x in range(100):

    writer.add_scalars(
        tag='y=x**2',  # 图像的唯一标识标签名
        scalar_value=x**2,  # 要记录的标量
        global_step=x,  # x轴
    )

    writer.add_scalars(
        main_tag='test_for',  # 图像的唯一标识标签名
        {'y=sinx':np.sin(x),'y=cosx':np.cos(x)},  # key是变量的tag,value
是变量的值
        x,  # x轴
    )
```

   - 直方图与多分位数折线图

```
writer.add_histogram(  # 统计直方图与多分位数折线图
    tag='y=x',  # 图像的唯一标识标签名
    values=np.random.normal(size=10000),  # x要统计的参数
    global_step=1  # y轴
    bins='tensorflow'  # 取直方图的bins
)
```

   - 图像

```
writer.add_image(  # 记录图像
    tag,  # 图像的唯一标识标签名
    img_tensor,  # 图像数据,注意尺度[0,1]*255,[0,1+)*1
    global_step=None,  # x轴
    dataformats='CHW'  # 数据形式,CHW/HWC/HW
)

torchvision.utils.make_grid(  # 制作网格图像
    tensor,  # 图像数据,BCHW形式
    nrow=8,  # 行数,列数自动计算
    padding=2,  # 间距
    normalize=False,  # 像数值标准化
    range=None,  # 标准化范围
    scale_each=False,  # 单张图维度标准化
    pad_value=0  # padding的像素值
)
```

- 计算图

```
writer.add_graph(  # 可视化模型计算图
    model,  # 模型
    input_to_model=None,  # 模型输入数据
    verbose=False  # 打印计算图
)
```

- 模型信息

```
# pip install torchsummary
torchsummary.summary(  # 查看模型信息,便于调试
    model,  # 模型
    input_size,  # 输入size
    batch_size=-1,  # 批大小
    device='cuda'  # cuda or cpu
)
```

4. 网址

cmd: `tensorboard --logdir-./runs`

八.Hook & CAM

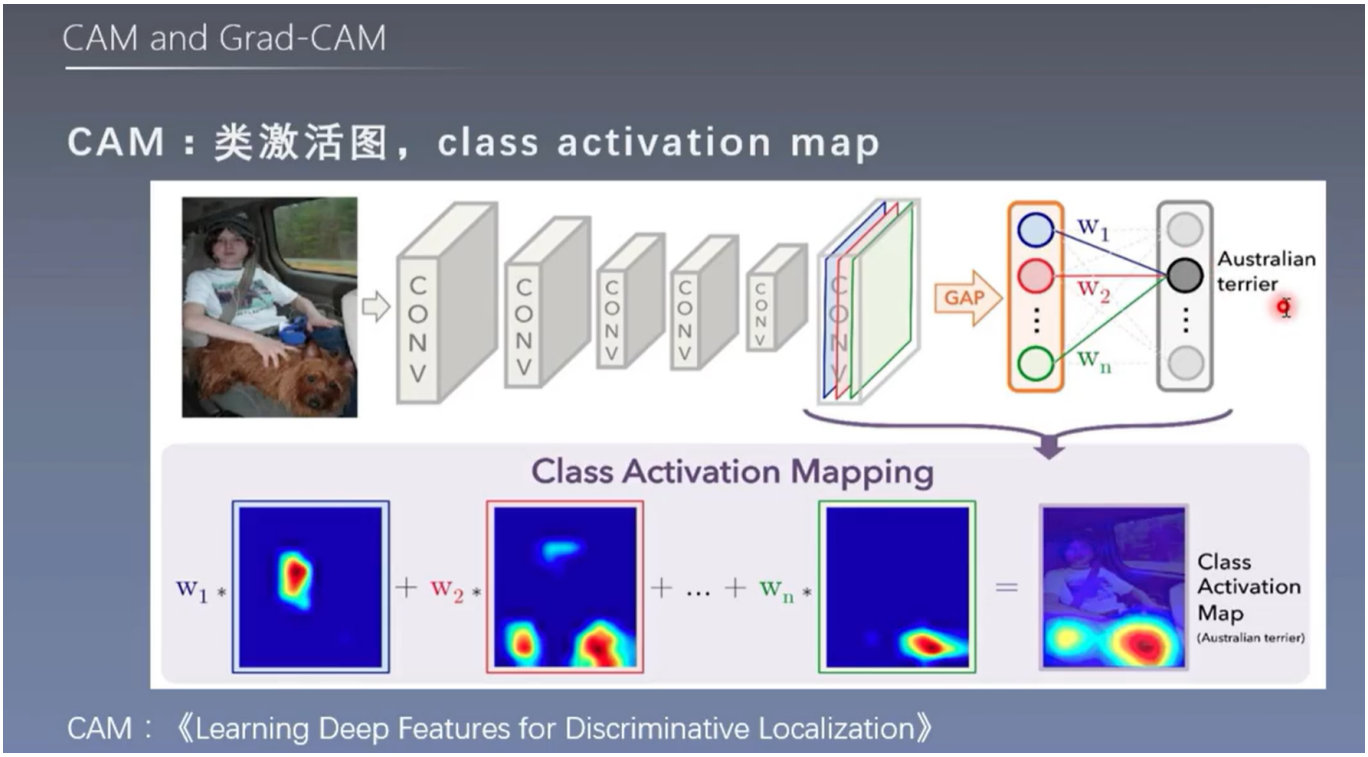
1.Hook

不改变原有功能,增加新的功能

声明

注册

2.CAM



十.Regularization

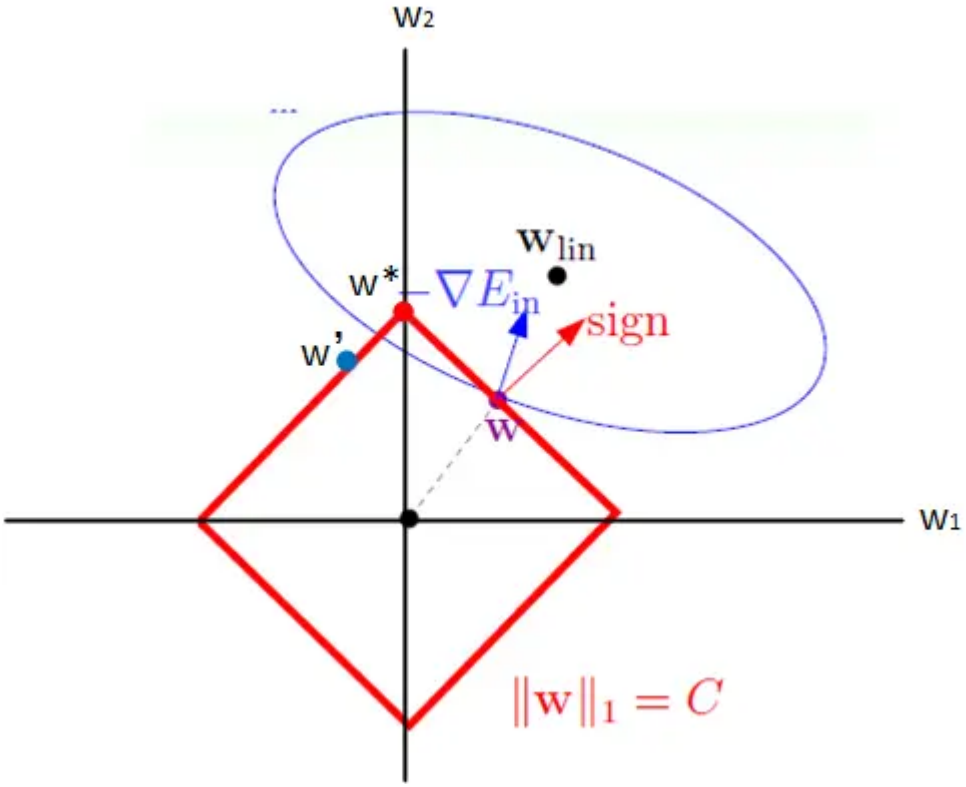
减小方差的策略(防止过拟合)

误差=偏差(算法)+方差(数据)+噪声(任意)

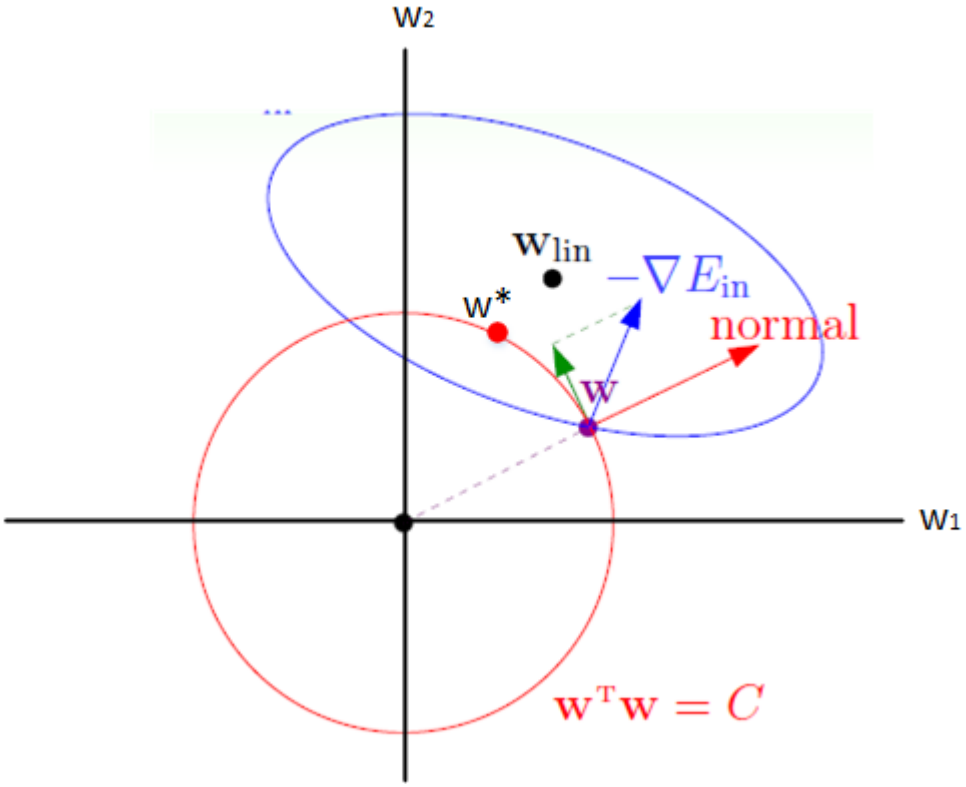
1.weight decay(权值衰减)

$Loss=Loss+Regularization Term$

- L1 Regularization Term: $sigma(|w|)$



- L2 Regularization Term: $sigma(w^2)$



L2 Regularization == weight decay(权值衰减)

原:$w=w-dLoss/dw$

L2:$Loss=Loss+k/2*sigma(w^2)$

$w=w*(1-k)-dLoss/dw$

相比之下w系数由1变为1-k,故称为权值衰减(k in range(0,1))

```
Optimizer(weight_decay=k)
```

2.Dropout(随机失活)

```
torch.nn.Dropout(p=0.5,inplace=False)
```

实现细节:

训练时权重均乘以1/(1-p)

(等价于测试时权重乘以(1-p))

3.Normalization(标准化)

ICS: 数据尺度变化

- Batch Normalization(BN)

```
torch.nn.BatchNorm2d(
)
```

- Layer Normalization(LN)

```
torch.nn.LayerNorm(
)
```

- Instance Normalization(IN)

```
torch.nn.InstanceNorm2d(
)
```

- Group Normalization(GN)

```
torch.nn.GroupNorm(
)
```

十一.Transfer Learning

1.保存与加载

1. torch.save(obj,f) # 对象(不止net),路径

   仅保存参数net.state_dict()

2. torch.load(f,map_loction) # 路径,cpu/cuda

   加载参数net.load_state_dict(state_dict)

2.断点续训练

```
check_point={
    'net_state_dict':net.state_dict(),
    'optimizer_state_dict':optimizer.state_dict(),
    'epoch':epoch
}

torch.save(check_point, path)

# ----------------------------------------------------------------------

check_point = torch.load(path)

net.load_state_dict(check_point['net_state_dict'])
optimizer.load_state_dict(check_point['optimizer_state_dict'])
start_epoch=check_point['epoch']
scheduler.last_epoch=start_epoch
```

3.模型微调(Model Finetune)

- 卷积层:特征提取features extractor
- 全连接层:分类classifier

1. 获取预训练模型参数
2. 加载模型
3. 修改输出层
4. 微调训练
   - 固定features extractor参数:

     requires_grad=False或lr=0
   - features extractor参数较小学习率:

     Optimizer([{'classifier':lr},{'features extractor':lr*0.01}])

十二.GPU

1.CPU & GPU

- CPU(Central Processing Unit,中央处理器)
- GPU(Graphics Processing Unit,图形处理器)

必须在相同设备上才能运算

2.转移设备

- tensor=tensor.to(dtype/'cuda'/'cpu')
- module.to(dtype/'cuda'/'cpu')

tensor会转移内存,module不会

3.多GPU并行运算

分发数据->并行运算->结果回收

```
torch.nn.DataParallel(
    module,  # 需要包装分发的模型
    device_ids=None,  # 可分发的GPU,默认为所有可见可用的GPU
    output_device=None,  # 结果输出设备,默认为主GPU
)
```

十三.Apply

1.图像分类

标签

- LeNet, 1998
- AlexNet, 2014
- VGGNet, 2012
- GoogLeNet, 2014
- ResNet, 2015

2.图像分割

像素级分类

- FCN, 2014
- U-Net, 2015
- DeepLab-v1, 2014
- DeepLab-v2, 2015

- DeepLab-v3, 2017
- DeepLab-v3+, 2018

3.目标检测

类别 & 边界框

> ***two-stage***

- R-CNN, 2014
- Fast R-CNN, 2015
- Faster R-CNN, 2015

> ***one-stage***

- YOLO, 2016
- SSD, 2016
- RetinaNet, 2017

4.生成对抗网络

Generator & Discriminator

- GAN, 2014
- DCGAN, 2015
- CycleGAN, 2017
- PixelDTGAN, 2016
- SRGAN, 2016
- Progressive GAN, 2017
- StackGAN, 2017
- Context Encoder, 2016
- Pix2Pix, 2016
- IcGAN, 2016

5.循环神经网络

不定长序列

- RNN
- LSTM

- .....